

Planejamento — Previsão de Carga Elétrica

Documento de planejamento técnico e organizacional do projeto para o Desafio Squad Pandas (Venture Studio). Foco em estrutura e especificação técnica. Não inclui cronograma nem prazos.

1. Visão geral

Aplicação web que **prevê a carga elétrica horária do dia seguinte** (day-ahead, 24h) por subsistema do Sistema Interligado Nacional, e mede a qualidade dessa previsão contra dois baselines: um modelo ingênuo e a previsão oficial do próprio ONS.

O projeto se encaixa na categoria **séries temporais e previsão** das diretrizes do desafio. A ciência de dados não é um enfeite: o modelo é servido pela aplicação e a avaliação é parte central do produto.

2. Objetivo

Entregar uma previsão day-ahead, horária, por subsistema, e demonstrá-la com rigor estatístico. O resultado não é "um gráfico de previsão" — é uma afirmação mensurável do tipo:

"O modelo erra X% (MAPE); o baseline ingênuo erra Y%; a previsão oficial do ONS erra Z% — todos avaliados sobre as mesmas horas de teste."

A existência de um **baseline real e honesto** (a programada do ONS) é o diferencial central do projeto.

3. Enquadramento e mapa para os critérios

Restrições do desafio respeitadas: entrega individual, custo zero (apenas free tier), e componente real de ciência de dados (não é CRUD).

Critério	Peso	Onde é endereçado
Componente de DS (profundidade e correção)	30%	Backtesting walk-forward sem vazamento, comparação contra ingênuo e ONS, escada de modelos, métricas com recortes
Aplicação ponta a ponta	25%	Next.js + rota <code>/api/forecast</code> servindo o modelo ao vivo, deploy na Vercel
Modelagem de dados no Supabase	15%	Schema versionado de 6 tabelas (incl. <code>model_runs</code> e <code>evaluations</code>)
Experiência de uso e clareza visual	15%	Painel com comparação de 3 linhas + KPIs + recortes de erro; página de metodologia
Uso inteligente do Cursor	10%	Pipeline Python, schema SQL e componentes do front escritos com assistência de IA
Apresentação e storytelling	5%	Demo ao vivo da <code>/api/forecast</code> ; narrativa "modelo × ONS"

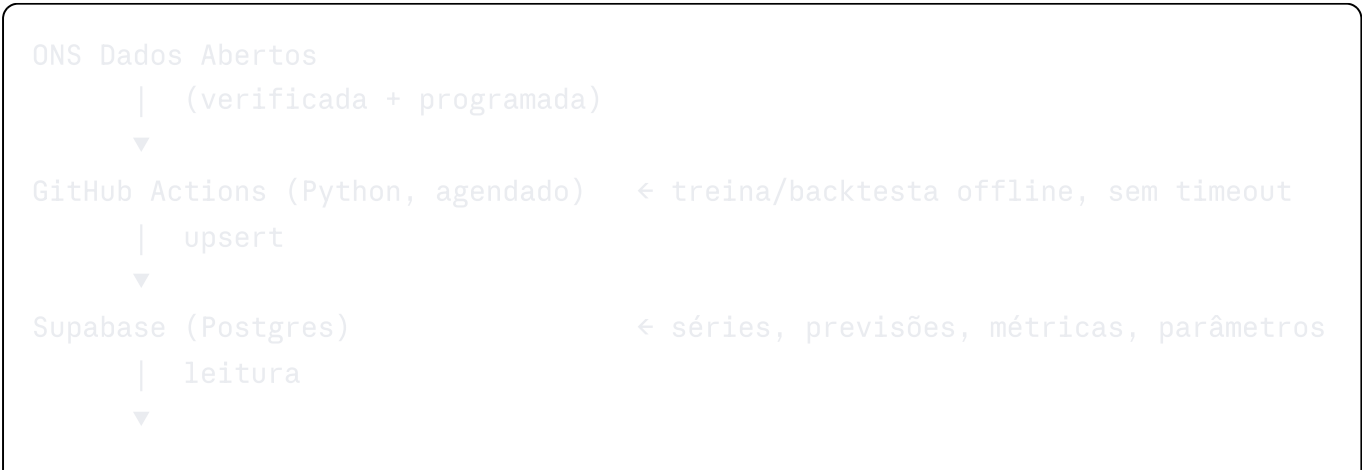
4. Arquitetura e stack

Princípio central

As bibliotecas de previsão são Python, e a Vercel roda funções serverless com duração curta (dezenas de segundos no Hobby, com erro 504 ao estourar). A solução é **separar treino de inferência**:

- **Treino e backtest** rodam fora da Vercel, num job Python agendado (GitHub Actions). Sem limite de tempo relevante.
- **Inferência** acontece no runtime e é leve (produto escalar, milissegundos) — cabe folgado no orçamento da Vercel.

Fluxo de dados (ponta a ponta)





Papel de cada stack

- **Cursor** — IDE assistida por IA. Escreve o pipeline Python, o schema SQL e os componentes/rotas do Next.js.
- **Supabase** — a espinha. Guarda séries históricas, previsões, métricas de avaliação e os parâmetros do modelo servido. Não usa pgvector.
- **Next.js** — front (gráficos e seletores) e a rota de inferência (`/api/forecast`).
- **Vercel** — deploy e URL pública. **Restrição absoluta: nunca treinar aqui.**
- **GitHub Actions** — CI/CD do próprio GitHub (exigido como repositório no desafio). É a "cola" legítima que roda o pipeline agendado; não é uma quinta plataforma de produto.

Restrições de free tier e como são tratadas

- **Vercel** — **timeout curto**: inferência pesada nunca roda na Vercel; o treino vai pro GitHub Actions.
- **Supabase** — **pausa após 7 dias sem requisição**: o job diário escreve no banco e funciona como keep-alive automático.
- **Supabase** — **500 MB de banco**: guarda-se dado tabular (séries + agregados), não imagens nem features intermediárias. O volume previsto (carga horária de 4 subsistemas por ~3 anos ≈ 100 mil linhas) cabe com folga.

5. Modelo de dados (Supabase)

Seis tabelas, enxutas e com propósito claro.

`subsystems` — dimensão de referência

coluna	tipo	nota
id	int (PK)	
codigo	text	casa com <code>cod_areacarga</code> da API do ONS
nome	text	SE/CO, S, NE, N

`load_actual` — carga verificada (alvo)

coluna	tipo	nota
id	bigint (PK)	
subsystem_id	int (FK)	
ts	timestamptz	instante horário
load_mw	numeric	MWmed
extracted_at	timestamptz	"as-of": quando o dado foi puxado

Chave única `(subsystem_id, ts)`. Índice **BRIN** em `(ts)` (minúsculo, ideal para dado append-only ordenado no tempo).

`load_official_forecast` — carga programada (baseline ONS)

coluna	tipo	nota
id	bigint (PK)	
subsystem_id	int (FK)	
ts	timestamptz	instante previsto
load_mw	numeric	MWmed
reference_date	date	dia em que o ONS programou

Separada de `load_actual` por causa do `reference_date`, que a verificada não possui.

`model_runs` — versionamento de modelo

coluna	tipo	nota
id	int (PK)	
model_name	text	ex.: <code>ridge_v1</code> , <code>lgbm_v2</code>
trained_at	timestamptz	
hyperparams	jsonb	
artifact	jsonb	coeficientes do Ridge para inferência em TS
git_commit	text	reprodutibilidade
train_start / train_end	date	janela de treino

predictions — saída do modelo

coluna	tipo	nota
id	bigint (PK)	
model_run_id	int (FK)	
subsystem_id	int (FK)	
target_ts	timestamptz	hora-alvo prevista
predicted_mw	numeric	

Versionada por **model_run_id** — histórico nunca é sobrescrito.

evaluations — métricas de avaliação

coluna	tipo	nota
id	bigint (PK)	
model_run_id	int (FK)	
subsystem_id	int (FK)	
predictor	text	modelo naive ons
metric	text	mape mae rmse
value	numeric	
horizon_h	int	horizonte avaliado

A coluna **predictor** torna a comparação de primeira classe: os três competidores ficam na mesma estrutura, prontos pro gráfico.

Decisões de design

- **Features engenheiradas não são persistidas** — são recalculadas no pipeline, economizando espaço.
- **RLS com política de leitura pública** nas tabelas de dados (dashboard é só-leitura).
- **Auth do Supabase é opcional** — não é necessária para a profundidade; só se houver um botão "admin" de retreino.

6. Pipeline de ingestão

Fontes e endpoints

Dois endpoints REST irmãos do ONS, mesma base (semi-horária, por área de carga), retornando JSON:

- Verificada (alvo): `https://apicarga.ons.org.br/prd/cargaverificada?dat_inicio=YYYY-MM-DD&dat_fim=YYYY-MM-DD&cod_areacarga=AREA`
- Programada (baseline ONS): `https://apicarga.ons.org.br/prd/cargaprogramada?dat_inicio=YYYY-MM-DD&dat_fim=YYYY-MM-DD&cod_areacarga=AREA`

Usar os dois da mesma API garante comparação justa (mesmo grão e base).

Estratégia de carga

- **Backfill (uma vez, ~3 anos):** para o histórico, usar os arquivos CSV (UTF-8, delimitado) da *Curva de Carga Horária* do ONS, agrupados por período — mais eficiente que milhares de chamadas.
- **Incremental (diário):** puxar uma **janela móvel dos últimos ~10 dias** de verificada e programada e fazer *upsert*. A janela móvel captura as reconsolidações do ONS (por isso `extracted_at` + chave única).

Regras de tratamento

1. **Grão:** agregar semi-horária → horária (média), para casar com o horizonte day-ahead horário.
2. **Fuso:** timestamps em horário de Brasília (UTC−3). Sem horário de verão no Brasil desde 2019 (sem saltos de DST). Gravar como `timestamp_tz` com offset explícito.
3. **Idempotência:** `INSERT ... ON CONFLICT (subsystem_id, ts) DO UPDATE` — reingestão nunca duplica.
4. **Buracos:** horas faltantes ficam explícitas (não inventar valor); tratamento de gaps é decidido na modelagem.
5. **Unidade:** MWmed, como o ONS entrega.

7. Features e modelagem

Definição do alvo e regra anti-vazamento

Previsão day-ahead: no fim do dia D, prever as 24h de D+1 usando **apenas o que já era conhecido em D**.

Regra de ouro: proibido usar defasagens menores que 24h. Usar a "carga da hora anterior" transformaria o modelo num preditor de 1 passo à frente — artificialmente

bom e injusto contra a programada do ONS (que é genuinamente day-ahead). O piso de 24h é o que torna a comparação válida.

Decisão: um modelo por subsistema (4 modelos). Perfis de carga diferem bastante entre regiões; separar é mais simples e mais preciso que um modelo único com dummies.

Lista de features (todas conhecidas em D)

- **Defasagens:** carga em $t-24h$, $t-48h$, $t-168h$ (mesma hora, semana anterior).
- **Estatísticas móveis** (até a emissão): média e desvio das últimas 24h e dos últimos 7 dias; média da mesma hora nos últimos N dias.
- **Calendário:** hora do dia, dia da semana, mês, fim de semana, feriado (lib `holidays`), véspera/emenda de feriado.
- **Sazonalidade cíclica (Fourier):** sen/cos da hora (período 24), do dia da semana (período 7) e do dia do ano (período 365).
- **Clima/temperatura:** fora do escopo inicial; primeira esticada se houver tempo.

Escada de modelos

1. **Baseline ingênuo sazonal:** $\hat{y}(t) = \text{carga}(t-168h)$. Piso a ser batido.
2. **Baseline ONS (programada):** a previsão oficial; a barra a perseguir.
3. **Ridge (linear regularizado)** sobre as features. Interpretável e rápido; coeficientes serializam para JSON (coluna `artifact`). **É o modelo servido** em TS pelo Next.js.
4. **Gradient boosting (LightGBM):** capta não-linearidades e interações. Costuma ser o campeão; **reportado como melhor resultado** na comparação.

Sem deep learning — volume e horizonte não justificam, o custo de treino é alto e não bateria um GBM bem-featurizado. Não sobre-engenheirar também é correção técnica.

Protocolo de backtesting — walk-forward

Validação cruzada aleatória (k-fold) é **proibida** em série temporal (embaralhar vaza o futuro no passado). Usa-se a janela deslizante de origem:

- Trava uma janela inicial de treino (~2 anos).
- Prevê o dia seguinte como em produção (day-ahead, só com dados até a emissão).
- Registra previsto $\times$ real; rola a origem à frente; re-treina; repete — varrendo os últimos ~6-12 meses como teste.
- **Janela expansível** (o treino cresce) por padrão; a deslizante de tamanho fixo é alternativa.

Duas regras de ouro:

- **Sem vazamento em nenhuma origem** — features só enxergam dados $\leq$ instante de emissão.

- **Os três preditores nos mesmos timestamps** — modelo, ingênuo e ONS avaliados sobre exatamente as mesmas horas. Tudo gravado em `evaluations`.

Métricas

- **MAPE (%)** — principal e intuitivo (sem risco de divisão por ~zero em carga agregada).
 - **MAE (MWmed)** — erro absoluto, robusto.
 - **RMSE (MWmed)** — pune erros grandes (o que dói na operação real).
 - **Recortes:** erro por hora do dia e por tipo de dia (útil × fim de semana × feriado) — mostra *onde* o modelo quebra.
 - **Skill score:** ganho percentual sobre cada baseline.
-

8. Camada de serviço e frontend

API e leitura de dados

- A maior parte das leituras ocorre em **Server Components** consultando o Supabase direto no servidor (anon key + RLS de leitura pública) — eficiente, sem rotas dedicadas.
- A única rota que *computa* é `/api/forecast`: dado subsistema + data-alvo, lê o `artifact` (coeficientes do Ridge) e os atuais necessários, monta o vetor de features **em TS** e faz $\hat{y} = \text{intercepto} + \sum w_i \cdot x_i$. Milissegundos, zero Python no runtime. É o que satisfaz "modelo servido pela aplicação".

Cuidado crítico — paridade de features: a engenharia de features em TS (serviço) precisa reproduzir exatamente a do Python (treino). Mitigação: manter a transformação simples e gravar casos de referência (entrada → previsão) do Python para um **teste de paridade** que afirma que a rota TS reproduz os mesmos números.

Segredos e segurança

- Credenciais do Supabase em Secrets do repositório, nunca commitadas.
- A chave `service_role` existe apenas dentro do GitHub Actions (servidor).
- O cliente Next.js usa apenas a `anon` + RLS de leitura. As duas chaves nunca se cruzam.

Telas

1. **Painel** — seletor de subsistema, comparação do dia (real × modelo × ONS), KPIs (MAPE do modelo, do ingênuo, do ONS, skill score) e recortes de erro (por hora, por tipo de dia).
2. **Avaliação aprofundada** — distribuição de resíduos e comparação Ridge × LightGBM × baselines lado a lado.

3. **Sobre / metodologia** — página curta espelhando o README (problema, fonte, features, validação, limitações).

O botão "prever dia seguinte" dispara a `/api/forecast` ao vivo — a prova concreta de modelo servido pela aplicação.

Biblioteca de gráficos

Recharts (React-nativo, leve, integra direto com o Next.js). Chart.js é alternativa equivalente.

9. Automação (GitHub Actions)

Um único workflow agendado com dois gatilhos:

```
yaml

on:
  schedule:
    - cron: '0 10 * * *' # diário (07h de Brasília)
  workflow_dispatch:      # disparo manual - essencial para a demo
```

Passos do job (Python): checkout → setup-python → instala deps → roda o pipeline. O pipeline ingere a janela móvel (upsert) → recalcula features → gera a previsão do dia seguinte → grava em `predictions`.

O **backtest completo + retreino** (gerando novo `model_runs` e `evaluations`) roda sob demanda via `workflow_dispatch` ao ajustar o modelo — não precisa rodar a cada execução diária.

Benefício colateral: como o job escreve no Supabase, conta como requisição ao banco e **funciona como keep-alive** contra a pausa de 7 dias.

Dois cuidados honestos: runs agendados podem atrasar sob carga — no dia da apresentação, disparar manualmente pelo `workflow_dispatch`; e o GitHub desabilita workflows agendados após 60 dias sem commit (irrelevante no horizonte do projeto, mas registrado).

10. Entregáveis

Conforme o desafio:

- **App no ar** — publicado na Vercel, com URL pública.
- **Repositório** — código versionado no GitHub.
- **Supabase** — projeto com schema modelado e dados.
- **README** — problema, fonte dos dados, abordagem técnica e limitações.

- **Demo** — apresentação com demonstração ao vivo (incluindo a `/api/forecast` em tempo real).

11. Riscos e mitigações

Risco	Mitigação
Setup da ingestão é o maior risco do projeto	Priorizar dado no Supabase antes de qualquer outra coisa; cortar escopo (menos subsistemas / histórico menor) se travar
API do ONS instável ou com limite	Pipeline grava tudo no Supabase; a demo nunca depende da API ao vivo
ONS reconsolida dados após publicação	Janela móvel de reingestão + <code>extracted_at</code> ("as-of")
Divergência de features Python × TS	Teste de paridade com casos de referência
Estouro dos 500 MB do Supabase	Guardar séries e agregados, não features intermediárias
Pausa do Supabase (7 dias)	Job diário do GitHub Actions como keep-alive

12. TODOs de setup (confirmar tocando a API)

Itens que só se resolvem na primeira chamada — não devem ser chutados:

- ☐ **Códigos de `cod_areacarga`**: confirmar se são os 4 subsistemas direto (SECO, S, NE, N) ou áreas mais finas a agregar. Passo zero: chamada exploratória para enumerar os códigos válidos.
 - ☐ **Nomes exatos dos campos** no JSON/CSV (coluna de carga, de instante, de área). Um `print` da primeira resposta resolve.
-

Princípio de execução

Fatiar fino e cedo: ter um fluxo ponta a ponta cru (dado → baseline → um gráfico numa página deployada) o quanto antes, e **melhorar** a partir dele, em vez de integrar tudo no final. A ciência de dados concentra o maior valor e o maior risco — recebe o foco principal.